Anubhav Agrawal

anubhavagr.mail@gmail.com | +91 7024880719 | Anubhav Agrawal | anubhavagr | anubhavagr.github.io

Summary

Machine Learning Engineer with **5+ years** of experience — **3+** of them full-time — building AI systems end-to-end: model R&D, real-time inference, *and* the production platform around them.

Technical Skills

Languages & Tools: Python, C++, Git, Linux, LaTeX
ML & Deep Learning: PyTorch, TensorFlow, OpenCV, Scikit-Learn, XGBoost, NumPy, Pandas
LLMs & RAG: OpenAI API, LangGraph, HuggingFace, Vector Search, Cross-Encoder Re-ranking, MongoDB
Inference & Serving: TensorRT, ONNX, INT8/FP16 Quantization, CUDA, FastAPI, mlx-lm, llama.cpp
Platform & MLOps: AWS (EC2), Docker, CI/CD, OpenTelemetry, Grafana LGTM (Loki, Tempo, Mimir)

Experience

Griphic — *Delhi*
AI Engineer II — *Jan 2026 – Present*

- Built the **adaptive-intelligence core** of an AI interviewer: a stateful **LangGraph** agent that scores responses, adapts **difficulty in real time**, and conditions every follow-up on retrieved context.
- Built the **RAG pipeline** — **3 parallel retrieval paths**, *hybrid keyword + dense search*, *cross-encoder re-ranking* over **512-d** embeddings — at **0.95 Recall@5** across **10K+ documents**.
- Cut query latency **2.5s → 0.8s** (**68%**) with context fusion and async orchestration; raised domain-query **F1 by 42%** on a held-out eval set; serves **500+ users** at **99.9% uptime**.
- Deployed the production system end-to-end — **Python/FastAPI** on **AWS EC2**, **MongoDB**, **OpenAI APIs** — with a **CI/CD pipeline** of automated **Docker** builds from commit to deploy.
- Set up observability: an **OpenTelemetry Collector** feeding logs, traces, and metrics to the **Grafana LGTM** stack (**Loki, Tempo, Mimir**), plus **LLM token-tracking and cost** dashboards for per-feature spend.
- Wrote the **evaluation suite** — recall, faithfulness, answer-relevance — and citation-grounded **LLM guardrails** that keep every answer traceable to source documents.

Innvolution Healthcare Pvt. Ltd. — *Bangalore*
***Machine Learning Engineer** & Team Lead* — *Sep 2023 – Dec 2025*

- Took **AIMAG**, an X-ray **super-resolution & denoising** product at **$200K+ ARR**, from dataset curation through model R&D, quantization, and release to **clinical validation**.
- Served the model in real time: **PyTorch → ONNX → TensorRT** with **INT8/FP16 quantization** behind a **FastAPI** **(C++)** runtime — **600+ FPS** on an RTX 4090, a **70% latency** cut, **<5% quality loss**, **35% lower VRAM**.
- Automated the **QCA** pipeline (segmentation → calibration → prediction) to finish in **<4 s** with a fine-tuned **U-Net++ (ResNet50 + attention)** at **0.94 Dice**; deployed for live cardiac-catheterization analysis.
- Co-invented **3 patent applications** in X-ray image enhancement: single-image blind super-resolution, DSA roadmap generation, and real-time fluoroscopy enhancement.
- Led the **ML team**, owning roadmap, review standards, and delivery from research prototypes through **clinical validation**.

Image Quality Engineer Intern — *Nov 2022 – May 2023*

- Built motion-compensated stitching & denoising for **DSA** sequences: **20% clarity gain** at **<2-px** alignment error.

Internships

ISRO — Space Applications Centre — NDVI forecasting on **AWiFS, MODIS, Sentinel** — Jun 2022 – May 2023
Enord (remote) — Multiprocessing file-processing scripts for edge devices (drones, Raspberry Pi) — Sep – Dec 2022
IIIT-NR — Smoking & spitting detection + dataset (**ResNet18**, transfer learning) for CCTV — May – Jun 2022

Projects

Inference Lab — LLM Serving Benchmarks on Apple Silicon — GitHub | Write-up
Reproducible benchmark suite: mlx-lm vs llama.cpp under a single fairness contract — *2026*

- Benchmarked **mlx-lm vs llama.cpp** on **Qwen2.5-7B-Instruct** (4-bit) under identical conditions — fixed prompts, greedy decoding, matched budgets — measuring **latency, throughput, and memory**: **45–56 tok/s** single-stream on a **MacBook Pro M4 Pro** (24 GB unified memory).
- Found why throughput stays **flat from 1 to 32 concurrent users**: requests serialize through a **per-instance lock** inside each serving engine.
- Showed the limit is **memory bandwidth**: a **multiprocessing K-instance dispatcher** gains only **+9% for 3× the memory**, because decoding already saturates the M4 Pro's ∼**273 GB/s** memory bus.

Education

Bhilai Institute of Technology, Durg — B.Tech in Computer Science Engineering — 2019 – 2023